

LangGraph

실무 가이드 · 지식 점검

프로젝트 투입을 위한 핵심 개념 · 패턴 · 코드 · 함정 정리

상태 기반 에이전트 오케스트레이션 런타임

LangGraph v1.x 기준 (2026년 6월) · 대상: 풀스택 / MLOps / DevOps 엔지니어

난이도: 실무 핵심 · 본문 한국어 + 코드/용어 영문

00 목차

1. LangGraph란 무엇이고 언제 쓰는가
2. 핵심 멘탈 모델과 실행 구조
3. State와 Reducer
4. 그래프 작성: 노드·엣지·조건부·Send
5. Command와 동적 라우팅·핸드오프
6. 영속성과 체크포인트
7. 휴먼 인 더 루프 (interrupt)
8. 스트리밍
9. 메모리: 단기 vs 장기(Store)
10. 프리빌트 에이전트와 미들웨어
11. 멀티 에이전트 아키텍처
12. Functional API
13. 내구성·에러 처리·운영·배포
14. 자주 하는 실수 모음
15. API 빠른 레퍼런스

지식 점검 퀴즈 (객관식 20문항)

정답 및 해설

1 LangGraph란 무엇이고 언제 쓰는가

LangGraph는 LLM 기반 에이전트와 멀티스텝 워크플로를 그래프로 모델링하는 오케스트레이션 런타임이다. LCEL 체인이 한 방향으로 흐르는 DAG라면, LangGraph는 순환(cycle)·분기·상태 공유·일시정지/재개를 1급 시민으로 다룬다. "모델이 도구를 호출 → 결과를 보고 다시 판단"하는 ReAct 루프, 사람 승인 단계, 장시간 작업의 중단·재개가 자연스럽게 표현된다.

LangChain과의 역할 분담 (v1.0 GA, 2025-10)

v1.0에서 경계가 공식화됐다. **LangChain** = 에이전트 프레임워크(모델·도구 추상화, 통합, 에이전트 루프), **LangGraph** = 오케스트레이션 런타임(내구 실행, 스트리밍, 휴먼인터루프, 영속성). 고수준 진입점 `langchain.agents.create_agent`도 내부적으로 LangGraph 엔진 위에서 동작한다. 2026년 6월 기준 최신 안정 버전은 LangGraph 1.2 계열이다.

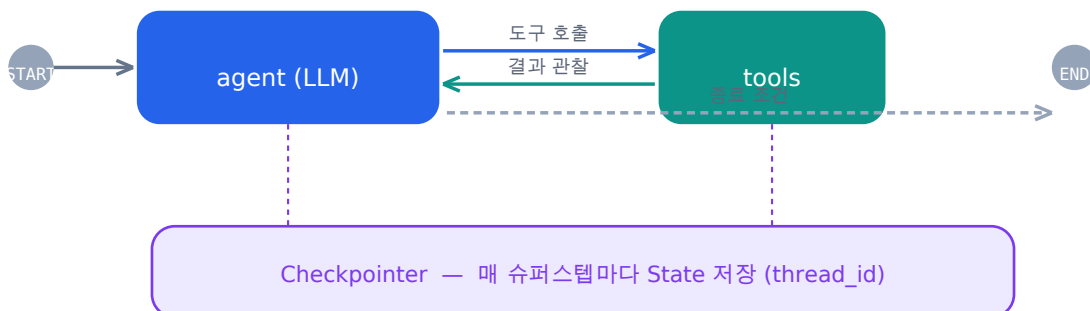
상황	권장
단일 프롬프트→응답, 단발성 RAG 1회 호출	체인 / 모델 직접 호출 (LangGraph 불필요)
도구 호출 루프, 조건 분기, 재시도	LangGraph StateGraph
사람 승인·검토 단계가 필요	LangGraph + interrupt
장시간/비동기 작업, 중단·재개·이력 추적	LangGraph + 체크포인트(영속성)
여러 전문 에이전트 협업	LangGraph 멀티 에이전트(supervisor/swarm)

한 줄 정의

LangGraph = "State를 공유하는 노드들의 그래프를, 매 단계 체크포인트로 저장하며 실행하는 런타임".

2 핵심 멘탈 모델과 실행 구조

- **State(상태)** — 그래프 전체가 공유하는 데이터. TypedDict/Pydantic 스키마로 정의.
- **Node(노드)** — 현재 State를 받아 **변경할 부분(부분 dict)**을 반환하는 함수.
- **Edge(엣지)** — 다음 노드를 결정. 고정 / 조건부 / 동적(Send).
- **START · END** — 가상의 진입점·종료점 상수.



▲ 전형적 ReAct 에이전트 그래프: agent ⇄ tools 루프 + 종료 엣지, 매 슈퍼스텝마다 체크포인트 저장.

실행 모델 — 슈퍼스텝(super-step)과 Pregel BSP

LangGraph는 Google Pregel에서 영감을 받은 **BSP(Bulk Synchronous Parallel)** 모델로 돈다. 한 슈퍼스텝에서 활성 노드들이 병렬로 실행되고, 모두 끝나면 그 결과로 State가 갱신된 뒤 다음 슈퍼스텝으로 넘어간다. 같은 슈퍼스텝에서 여러 노드가 같은 키를 쓰면 **Reducer**가 병합을 결정한다. 체크포인트는 **슈퍼스텝 경계마다** 저장된다 — 이것이 중단·재개·시간여행의 토대다.

3 State와 Reducer

State의 각 키는 기본적으로 **덮어쓰기(last-write-wins)**로 갱신된다. 누적이 필요하다면 **Annotated**로 **Reducer** 함수를 붙인다. 메시지 리스트에는 거의 항상 `add_messages`를 쓴다 — append뿐 아니라 **같은 ID 메시지의 갱신·중복 제거**까지 처리한다.

```
from typing import Annotated, TypedDict
import operator
from langgraph.graph.message import add_messages
from langchain_core.messages import AnyMessage

class State(TypedDict):
    user_id: str # default: overwrite
    messages: Annotated[list[AnyMessage], add_messages] # append + merge by id
    visited: Annotated[list[str], operator.add] # plain list append

def node(state: State) -> dict:
    return {"messages": [("assistant", "hi")], "visited": ["node"]} # return delta only
```

입력·출력 스키마를 본문 State와 분리할 수도 있다(`StateGraph(State, input=In, output=Out)`). 내부 작업용 필드를 외부 API 계약에서 숨길 때 유용하다.

자주 하는 실수

노드에서 State **전체**를 반환하지 말 것 — 변경분만 반환(전체 반환 시 Reducer와 충돌).
리스트 누적인데 Reducer를 안 붙이면 매번 덮어써진다 → `add_messages` 또는 `operator.add`.

4 그래프 작성: 노드·엣지·조건부·Send

`StateGraph`는 빌더다. 노드·엣지 등록 후 반드시 `.compile()`로 실행 그래프를 만든다. 컴파일 결과는 `invoke` / `ainvoke` / `stream` / `astream`을 지원한다.

```
from langgraph.graph import StateGraph, START, END

b = StateGraph(State)
b.add_node("agent", agent_node)
b.add_node("tools", tool_node)
b.add_edge(START, "agent")
b.add_conditional_edges("agent", route, {"call": "tools", "done": END}) # branch
b.add_edge("tools", "agent") # loop
graph = b.compile(checkpointer=cp)
```

- `add_edge(A, B)` — A 후 항상 B. 출발을 리스트로 주면 모두 끝날 때까지 대기(fan-in).
- `add_conditional_edges(A, fn, map)` — `fn(state)` 반환값으로 다음 노드 선택.
- `Send` — 런타임에 결정되는 **동적 팬아웃(map-reduce)**. 같은 노드를 N개 입력으로 병렬 실행.

```
from langgraph.types import Send
# a conditional edge returns a list of Send -> run "worker" once per item, in parallel
```

```
def fan_out(state):
    return [Send("worker", {"item": x}) for x in state["items"]]
b.add_conditional_edges("split", fan_out, ["worker"])
```

5 Command와 동적 라우팅·핸드오프

분기는 두 방식이 있다. (1) **조건부 엣지**: 라우팅 함수가 다음 노드 이름만 반환. (2) **Command**: 노드가 **State 업데이트 + 다음 행선지를 동시에** 반환. 멀티 에이전트 핸드오프에 특히 유용하다.

```
from langgraph.types import Command
from typing import Literal

def agent(state: State) -> Command[Literal["tools", "__end__"]]:
    ai = llm.invoke(state["messages"])
    goto = "tools" if ai.tool_calls else "__end__"
    return Command(update={"messages": [ai]}, goto=goto) # update + move together

# from a subgraph node, jump to a node in the PARENT graph (handoff)
# return Command(goto="alice", graph=Command.PARENT)
```

조건부 엣지 vs Command

단순 분기 → 조건부 엣지(가독성). 상태 변경과 이동을 묶거나 부모 그래프로 점프(graph=Command.PARENT)해야 하면 → Command.

6 영속성과 체크포인트

체크포인트는 매 슈퍼스텝마다 그래프 상태를 StateSnapshot으로 저장한다. 이것이 LangGraph의 핵심 차별점인 **내구성** 있는 **실행**·중단/재개·시간여행 디버깅을 가능하게 한다. 모든 실행은 config={'configurable': {'thread_id': ...}}로 스레드에 묶이며, 같은 thread_id면 저장된 상태 위에서 이어 실행된다.

```
from langgraph.checkpoint.postgres import PostgresSaver
with PostgresSaver.from_conn_string("postgresql://...") as cp:
    cp.setup() # create schema once
    graph = b.compile(checkpointer=cp)

cfg = {"configurable": {"thread_id": "user-42"}}
graph.invoke({"messages": [("user", "hi")]}, cfg)
snap = graph.get_state(cfg) # current StateSnapshot
history = list(graph.get_state_history(cfg)) # time travel
graph.update_state(cfg, {"messages": [("user", "edit")]} # human edits state
```

체크포인트	용도	영속	비고
InMemorySaver	개발·테스트	휘발성	프로세스 종료 시 소멸
SqliteSaver	로컬·단일 인스턴스	파일	경량 영속
PostgresSaver / Async	운영 표준	DB	다중 인스턴스/스케일아웃

durability 모드 (v1)

invoke/stream에 durability를 줄 수 있다: 'exit'(끝날 때만 저장·최속), 'async'(다음 스텝과 병렬 저장·기본 절충), 'sync'(매 스텝 동기 저장·최안전). 지연과 내구성의 트레이드오프를 상황에 맞게 선택.

7 휴먼 인 더 루프 (interrupt)

결제·파일 쓰기·SQL 실행 같은 민감한 액션 전 사람 승인을 받고 싶을 때, 노드 안에서 `interrupt(payload)`를 호출하면 그래프가 **그 지점에서 멈추고** 상태를 체크포인트에 저장한다. 사람이 검토 후 `Command(resume=value)`로 다시 `invoke`하면 **바로 그 노드부터** 재개되며 `resume` 값이 `interrupt()`의 반환값이 된다(체크포인트 필수).

```
from langgraph.types import interrupt, Command

def approval(state: State):
    decision = interrupt({"action": state["pending_action"]}) # pause here
    if decision == "approve":
        return {"status": "executed"}
    return {"status": "rejected"}

graph.invoke(inputs, cfg) # 1) pauses at interrupt
graph.invoke(Command(resume="approve"), cfg) # 2) resume with human input
```

패턴	방법
승인/거부(Approve/Reject)	interrupt로 멈춤 → resume에 결정 전달
수정 후 진행(Edit)	interrupt로 제안 노출 → resume에 수정값 전달
상태 직접 개입	graph.update_state(cfg, {...}) 후 재개
정적 중단(디버깅)	compile(interrupt_before=["node"])로 항상 경계에서 멈춤

8 스트리밍

`graph.stream()/astream()`으로 진행 상황을 실시간 방출한다. `stream_mode`로 무엇을 흘릴지 정하며 리스트로 동시 사용 가능. 토큰·중간 산출물을 더 세밀히 보려면 `astream_events()`를 쓴다.

stream_mode	내용	주 용도
"values"	매 스텝 이후 전체 State 스냅샷	전체 상태 추적
"updates"	각 노드가 만든 변경분만	가장 흔함(진행 로그)
"messages"	LLM 토큰을 토큰 단위로	채팅 UI 타자 효과
"custom"	노드 안 <code>get_stream_writer()</code> 로 임의 방출	진행률·중간 신호
"debug"	가장 상세한 실행 이벤트	디버깅

```
for mode, chunk in graph.stream(inputs, cfg, stream_mode=["updates", "messages"]):
    if mode == "messages":
        token, meta = chunk
        print(token.content, end="") # token streaming
    else:
        print(chunk) # {node_name: delta}
```

9 메모리: 단기 vs 장기(Store)

단기 기억은 체크포인트가 관리하는 **thread 내부 State**로 한 대화 세션 안에서만 유지된다. 장기 기억은 스레드 경계를 넘어 지속돼야 하므로 별도 **Store(BaseStore)**에 저장한다. Store는 namespace + key로 JSON 문서를 보관하며 put / get / search(시맨틱 검색 지원)를 제공한다.

```
from langgraph.store.memory import InMemoryStore # prod: PostgresStore
store = InMemoryStore()
graph = b.compile(checkpointer=cp, store=store)

def node(state: State, *, store): # store is injected
    ns = ("memories", state["user_id"])
    store.put(ns, "pref", {"lang": "ko"}) # long-term write
    hits = store.search(ns, query="language preference") # search
    return {}
```

구분	저장소	범위	용도
단기	thread State (체크포인트)	같은 thread_id	대화 맥락/중간 상태
장기	Store (BaseStore)	user 등 namespace 공유	사용자 선호·사실·요약

10 프리빌트 에이전트와 미들웨어

밑바닥부터 짜는 대신 검증된 ReAct 에이전트를 한 줄로 만든다. v1.0부터 `langgraph.prebuilt.create_react_agent`는 **deprecated**되고 `langchain.agents.create_agent`(미들웨어 시스템 포함)가 권장 진입점이다 — 둘 다 LangGraph 런타임 위에서 돈다.

```
from langchain.agents import create_agent # v1 recommended entrypoint
agent = create_agent(model="openai:gpt-4.1", tools=[search, calc])
agent.invoke({"messages": [("user", "convert the currency")]})
```

- **ToolNode** — 모델이 요청한 tool_calls를 실행해 ToolMessage로 되돌리는 프리빌트 노드.
- **tools_condition** — tool_calls가 있으면 tools로, 없으면 종료로 라우팅하는 프리빌트 조건.
- **미들웨어** — create_agent에서 도구 호출 전후로 끼어들어 승인(HITL)·요약·가드레일·재시도 등을 주입.

11 멀티 에이전트 아키텍처

패턴	구조	장점	패키지
Supervisor	중앙 오케스트레이터가 호출 결정	통제·관측 쉬움	langgraph-supervisor
Swarm	에이전트끼리 직접 제어권 이양	자율·분산	langgraph-swarm
Handoff	Command(goto, graph=PARENT)로 이양	유연	□□(Command)
Subgraph	컴파일 그래프를 부모의 노드로 삽입	모듈화·재사용	□□

핸드오프 시 **어떤 메시지를 다음 에이전트에 넘길지**(컨텍스트 설계)가 품질을 좌우한다. 기본 핸드오프 도구는 전체 메시지 이력을 넘기므로, 토큰·혼선을 줄이려면 전달 범위를 명시적으로 좁히는 것이 좋다. 서브그래프는 부모와 State 스키마가 다르면 입출력 변환 함수로 감싸 연결한다.

12 Functional API

그래프를 명시적으로 그리는 대신 **일반 파이썬 함수·반복문·조건문**으로 워크플로를 작성하는 대안 API. Graph API와 **같은 런타임**을 공유하므로 체크포인팅·중단·스트리밍을 그대로 누린다. 단순 구조에 적합하며 둘을 혼용할 수 있다.

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def summarize(doc: str) -> str:
    return llm.invoke(f"summarize: {doc}").content

@entrypoint(checkpointer=InMemorySaver())
def workflow(docs: list[str]) -> list[str]:
    futures = [summarize(d) for d in docs] # run in parallel
    return [f.result() for f in futures]
```

- **@task** — 비동기 실행 가능한 작업 단위. future 반환.
- **@entrypoint** — 워크플로 시작점. 내부에서 task 호출 + 일반 제어문으로 흐름 구성.
- 명시적 State/Reducer 선언 불필요(상태가 함수 스코프에 갇힘).

13 내구성·에러 처리·운영·배포

- **멱등성·재시도** — 노드는 재실행될 수 있으니 부수효과를 멱등하게. 노드별 RetryPolicy 설정 가능.
- **관측성** — LangSmith로 트레이스/토큰/지연/비용 추적. 환경변수만 켜면 자동 계측.
- **배포** — LangGraph Server/Platform이 그래프를 API로 노출하고 스레드·스토어·스케줄을 제공. 직접 FastAPI 래핑도 가능.
- **컨텍스트 관리** — 메시지 누적으로 토큰이 폭발하지 않게 트리밍/요약 미들웨어를 둔다.
- **비동기** — I/O 바운드 노드는 `async def + ainvoke/astream`.
- **결정성·테스트** — 노드는 가능한 순수 함수로. `get_state_history`로 특정 체크포인트에서 분기 재현.

14 자주 하는 실수 모음

증상	원인	해결
리스트가 매번 덮어써짐	Reducer 미설정	Annotated + add_messages/operator.add
interrupt가 동작 안 함	체크포인트 미설정	compile(checkpointer=...) 필수
재개가 처음부터 다시 시작	thread_id 불일치/누락	동일 config로 invoke
상태가 세션 간 안 남음	단기/장기 혼동	장기 데이터는 Store에 저장
한 스텝 두 노드가 같은 키 덮음	동시 쓰기 충돌	Reducer로 병합 규칙 정의
토큰 비용 폭증	메시지 무한 누적	트리밍/요약 미들웨어
프리빌트 deprecated 경고	create_react_agent 사용	create_agent로 이전

15 API 빠른 레퍼런스

목적	코드
그래프 정의	<code>StateGraph(State)</code>
노드/엣지	<code>add_node</code> / <code>add_edge</code> / <code>add_conditional_edges</code>
동적 팬아웃	<code>Send("node", payload)</code>
컴파일	<code>b.compile(checkpointer=cp, store=store)</code>
실행	<code>graph.invoke(inp, cfg)</code> / <code>stream</code> / <code>ainvoke</code>
상태 조회/수정	<code>get_state</code> / <code>get_state_history</code> / <code>update_state</code>
라우팅+이동	<code>Command(update=..., goto=...)</code>
휴먼인터럽트	<code>interrupt(payload)</code> / <code>Command(resume=...)</code>
스트리밍	<code>stream_mode = values updates messages custom debug</code>
프리빌트 에이전트	<code>langchain.agents.create_agent(model, tools)</code>

LangGraph는 빠르게 진화합니다. 본 문서는 2026-06 시점 v1.x 기준이며, 정확한 시그니처는 docs.langchain.com과 설치된 버전의 릴리스 노트로 최종 확인하세요.

Q 지식 점검 퀴즈 (객관식 20문항)

정답·해설은 다음 장에 있습니다. 보기 중 하나를 고르세요.

Q1. v1.0 기준 LangChain과 LangGraph의 역할 분담으로 가장 정확한 것은?

- A. LangChain은 런타임, LangGraph는 모델 추상화
- B. LangChain은 에이전트 프레임워크, LangGraph는 오케스트레이션 런타임
- C. 둘은 독립적이라 함께 못 쓴다
- D. LangGraph가 LangChain을 완전히 대체했다

Q2. 노드(Node) 함수가 반환해야 하는 것은?

- A. State 전체를 복사한 dict
- B. 다음 노드 이름 문자열만
- C. State에서 변경할 부분(부분 dict)
- D. 반환값 없음

Q3. messages에 add_messages Reducer를 쓰는 핵심 이유는?

- A. 항상 덮어쓰기 위해
- B. append + 같은 ID 메시지 병합·중복 제거
- C. 암호화
- D. 토큰 자동 축소

Q4. StateGraph 빌더를 실행하려면 먼저 무엇을 하나?

- A. .run()
- B. .compile()
- C. .build()
- D. 바로 invoke

Q5. 체크포인트는 언제 상태를 저장하는가?

- A. 그래프가 끝났을 때만
- B. 매 슈퍼스텝 경계마다
- C. 수동 save 호출 시
- D. 1초마다

Q6. 같은 대화를 이어서 실행되게 묶는 식별자는?

- A. run_id
- B. session_token
- C. thread_id
- D. checkpoint_ns

Q7. 운영·다중 인스턴스 스케일아웃에 가장 적합한 체크포인트는?

- A. InMemorySaver
- B. SqliteSaver
- C. PostgresSaver
- D. 체크포인트 없음

Q8. 노드 안에서 interrupt()를 호출하면?

- A. 예외로 영구 종료
- B. 그 지점에서 멈춰 상태 저장, Command(resume=)로 그 노드부터 재개
- C. 다음 노드로 즉시 진행
- D. State 초기화

Q9. LLM 토큰을 채팅 UI에 한 글자씩 흘리는 stream_mode는?

- A. "values"
- B. "updates"
- C. "messages"
- D. "debug"

Q10. 각 노드의 '변경분만' 받고 싶을 때 적절한 stream_mode는?

- A. "values"
- B. "updates"
- C. "custom"
- D. "messages"

Q11. 단기/장기 기억 저장 위치를 바르게 짚지은 것은?

- A. 단기=Store, 장기=체크포인트
- B. 단기=thread State(체크포인트), 장기=Store(BaseStore)
- C. 둘 다 체크포인트
- D. 둘 다 Store

Q12. Command 객체에 대한 설명으로 옳은 것은?

- A. State 업데이트와 다음 행선지(goto)를 동시에 표현
- B. 로깅 전용
- C. 컴파일 단계에서만 사용
- D. 조건부 엣지를 못 대체

Q13. v1.0에서 권장되는 ReAct 에이전트 생성 진입점은?

- A. langgraph.prebuilt.create_react_agent
- B. langchain.agents.create_agent
- C. AgentExecutor
- D. LLMChain

Q14. Supervisor 멀티 에이전트 패턴 설명으로 옳은 것은?

- A. 에이전트끼리 직접 제어권 이양(분산)
- B. 중앙 오케스트레이터가 전문 에이전트 호출 결정
- C. 단일 에이전트
- D. 사람이 매 스텝 수동 라우팅

Q15. Functional API(@entrypoint/@task)에 대해 옳은 것은?

- A. 별도 런타임 사용
- B. State/Reducer 선언 없이 일반 파이썬 제어문으로 작성, 체크포인팅·스트리밍 그대로
- C. 체크포인팅 미지원
- D. 동기 코드만 가능

Q16. 런타임에 항목 수만큼 같은 노드를 동적 병렬 실행(map-reduce)하는 메커니즘은?

- A. add_edge
- B. Send
- C. interrupt
- D. ToolNode

Q17. 서브그래프 노드가 부모 그래프의 노드로 점프하려면?

- A. Command(goto='x', graph=Command.PARENT)
- B. add_edge(START,'x')
- C. interrupt('x')
- D. END 반환

Q18. 사람이 멈춘 그래프의 상태를 직접 수정한 뒤 재개하려면?

- A. graph.reset()
- B. graph.update_state(cfg, {...})
- C. compile() 재호출
- D. 새 thread_id 발급

Q19. durability='exit'와 'sync'의 차이로 옳은 것은?

- A. exit=매 스텝 저장, sync=종료 시 저장
- B. exit=종료 시에만 저장(빠름), sync=매 스텝 동기 저장(안전)
- C. 둘 다 동일
- D. exit는 체크포인트 불필요

Q20. 모델의 tool_calls 유무로 tools 노드/종료를 라우팅하는 프리빌트는?

- A. tools_condition
- B. add_messages
- C. InMemoryStore
- D. Send

A

정답 및 해설

- Q1** 정답: B
v1.0(2025-10)에서 경계 공식화. LangChain=에이전트 프레임워크, LangGraph=오케스트레이션 런타임. 함께 쓰도록 설계.
- Q2** 정답: C
노드는 변경할 부분만 부분 dict로 반환. 전체 반환은 Reducer 의도와 충돌.
- Q3** 정답: B
add_messages는 append + 동일 ID 메시지 갱신/중복 제거를 처리하는 메시지 전용 Reducer.
- Q4** 정답: B
StateGraph는 빌더. .compile() 후에야 invoke/stream 가능.
- Q5** 정답: B
BSP 모델에서 체크포인트는 슈퍼스텝 경계마다 저장 — 중단/재개·시간여행의 토대.
- Q6** 정답: C
config.configurable.thread_id가 대화를 묶는 기본 키. 같은 값이면 저장 상태 위에서 이어 실행.
- Q7** 정답: C
PostgresSaver(Async 포함)가 운영·다중 인스턴스 표준. InMemory=휘발성, Sqlite=단일 노드.
- Q8** 정답: B
interrupt()는 그 지점에서 멈춰 상태를 저장. 이후 Command(resume=값)로 그 노드부터 재개되고 값이 interrupt() 반환값이 된다. 체크포인트 필수.
- Q9** 정답: C
"messages" 모드가 토큰 단위 스트리밍 제공(채팅 타자 효과).
- Q10** 정답: B
"updates"=각 노드 변경분만. "values"=매 스텝 전체 State 스냅샷.
- Q11** 정답: B
단기=thread State(체크포인트, 같은 thread에서만), 장기=Store(namespace+key, 스레드 넘어 공유).
- Q12** 정답: A
Command는 update(상태 변경)+goto(다음 노드)를 함께 표현. graph=Command.PARENT로 부모 그래프 점프(핸드오프).
- Q13** 정답: B
v1.0부터 langchain.agents.create_agent가 권장(미들웨어 포함). create_react_agent는 deprecated. 둘 다 LangGraph 런타임 위.
- Q14** 정답: B
Supervisor=중앙 오케스트레이터가 호출 결정(통제·관측 쉬움). 직접 이양은 Swarm.
- Q15** 정답: B
Functional API는 Graph API와 같은 런타임 공유. State/Reducer 선언 없이 일반 제어문 사용, 체크포인트·중단·스트리밍 그대로, async 지원.

Q16 정답: B

Send가 런타임 결정 동적 팬아웃(map-reduce). 조건부 엣지가 Send 리스트를 반환해 같은 노드를 N개 병렬 실행.

Q17 정답: A

서브그래프에서 Command(goto=..., graph=Command.PARENT)로 부모 그래프 노드로 이동(핸드오프).

Q18 정답: B

graph.update_state(cfg, {...})로 체크포인트 상태를 직접 수정 후 동일 config로 재개.

Q19 정답: B

'exit'=실행 종료 시에만 저장(가장 빠름), 'sync'=매 스텝 동기 저장(가장 안전). 'async'는 그 중간.

Q20 정답: A

tools_condition은 tool_calls 유무로 tools 노드/종료를 라우팅하는 프리빌트 조건 함수.